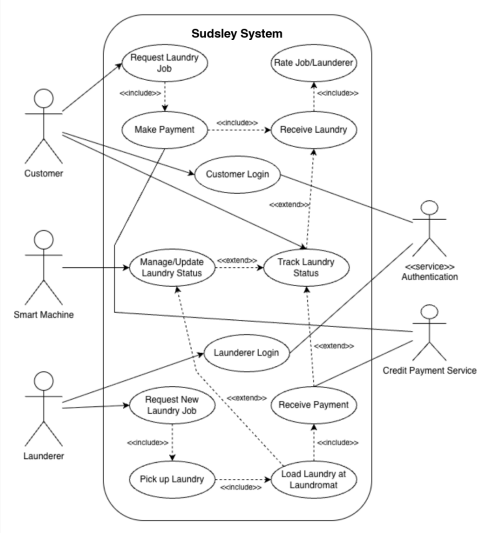
System Design

High-level and component design

Vincent Tran

High-level design

An example high-level design



Clients

There is a mobile app for each type of user.

- **Customer App** and **Launderer App**
- Distinct purposes, slightly different API calls
- The frontend team builds them

Backend services

- **Job Service.** The “brain”: orchestrates the whole laundry process, from job creation and launderer matching to tracking the job’s status. Its main purpose is to call and notify other services, so it is often called an orchestrator.

Backend services

- **Job Service.** The “brain”: orchestrates the whole laundry process, from job creation and launderer matching to tracking the job’s status. Its main purpose is to call and notify other services, so it is often called an orchestrator.
- **Payment Service.** Charges customers and pays launderers, through the Payment Provider.

Backend services

- **Job Service.** The "brain": orchestrates the whole laundry process, from job creation and launderer matching to tracking the job's status. Its main purpose is to call and notify other services, so it is often called an orchestrator.
- **Payment Service.** Charges customers and pays launderers, through the Payment Provider.
- **Location Service.** Tracks launderers in real time; gets ETAs and routes from the Maps Provider.

Backend services

- **Job Service.** The "brain": orchestrates the whole laundry process, from job creation and launderer matching to tracking the job's status. Its main purpose is to call and notify other services, so it is often called an orchestrator.
- **Payment Service.** Charges customers and pays launderers, through the Payment Provider.
- **Location Service.** Tracks launderers in real time; gets ETAs and routes from the Maps Provider.
- **Machine Service.** Finds available machines at Sudsley locations and remotely starts and monitors cycles through the Smart Machine API.

Backend services

- **Job Service.** The "brain": orchestrates the whole laundry process, from job creation and launderer matching to tracking the job's status. Its main purpose is to call and notify other services, so it is often called an orchestrator.
- **Payment Service.** Charges customers and pays launderers, through the Payment Provider.
- **Location Service.** Tracks launderers in real time; gets ETAs and routes from the Maps Provider.
- **Machine Service.** Finds available machines at Sudsley locations and remotely starts and monitors cycles through the Smart Machine API.
- **User Service.** Customer and launderer profile data, separate from authentication: saved addresses, preferences, aggregated ratings.

Backend services

- **Job Service.** The "brain": orchestrates the whole laundry process, from job creation and launderer matching to tracking the job's status. Its main purpose is to call and notify other services, so it is often called an orchestrator.
- **Payment Service.** Charges customers and pays launderers, through the Payment Provider.
- **Location Service.** Tracks launderers in real time; gets ETAs and routes from the Maps Provider.
- **Machine Service.** Finds available machines at Sudsley locations and remotely starts and monitors cycles through the Smart Machine API.
- **User Service.** Customer and launderer profile data, separate from authentication: saved addresses, preferences, aggregated ratings.
- **Notification Service.** Real-time push notifications to customers and launderers about job updates.

External services

- **Payment Provider** (e.g., Square, Stripe). Processes payments and payouts securely. We store no financial information.

External services

- **Payment Provider** (e.g., Square, Stripe). Processes payments and payouts securely. We store no financial information.
- **Maps Provider** (e.g., Google Maps). Mapping, geocoding, route planning.

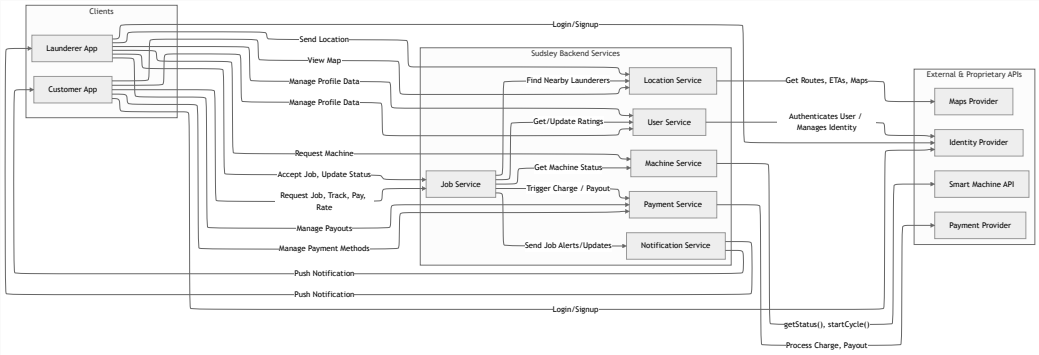
External services

- **Payment Provider** (e.g., Square, Stripe). Processes payments and payouts securely. We store no financial information.
- **Maps Provider** (e.g., Google Maps). Mapping, geocoding, route planning.
- **Identity Provider** (e.g., Auth0, Okta, Cognito). Sign-up, login, and credentials. We store no passwords or credentials.

External services

- **Payment Provider** (e.g., Square, Stripe). Processes payments and payouts securely. We store no financial information.
- **Maps Provider** (e.g., Google Maps). Mapping, geocoding, route planning.
- **Identity Provider** (e.g., Auth0, Okta, Cognito). Sign-up, login, and credentials. We store no passwords or credentials.
- **Smart Machine API.** Our own proprietary API for controlling the machines. "External" in the sense that we treat it as a black box that works.

Sudsley high-level design



Component-level design

From floor plan to schematic

- **High-level design** is an architect's floor plan of a house: the kitchen, the bedrooms, the bathrooms.
- **Component-level design** is the schematic for just the kitchen: exactly where the sink, stove, and dishwasher go, and how their plumbing and wiring connect.

Define the component-level design

After the high-level (macro) design comes the component-level (micro) design.

- Zoom in on one component from the high-level diagram: a microservice, a client app, or an external API.
- Move from the *what* (the components) to the *how* (their implementation details).

A component-level design:

- defines specific responsibilities,
- describes the internal logic, and
- specifies interactions and dependencies: which services it calls and which external dependencies it relies on.

Component architecture questions

1. What exact data fields should it store?
2. Does it need an API? If so, what endpoints?
3. What latency is acceptable?
4. How consistent should it be?
5. How much storage is needed?
6. How many read vs. write requests?
7. Is the data structured and relational, or more flexible?
8. How will the service scale?
9. Does it need a cache?
10. Is it stateless?
11. Does it need rate limiting?
12. What key metrics should it log?
13. Does it have external dependencies? What are the limits?

Let's design the User Service.

User Service: data, API, latency

Question	What it decides	User Service
1. What data fields should it store?	The data model: what this service owns, separate from data other services manage.	
2. Does it need an API?	The service's contract: what other services and the apps can call.	
3. What latency is acceptable?	The speed target, which drives database and caching choices.	

User Service: data, API, latency

Question	What it decides	User Service
1. What data fields should it store?	The data model: what this service owns, separate from data other services manage.	Customer: `user_id`, `display_name`, `saved_address`, `laundry_preferences`. Launderer: `user_id`, `display_name`, `average_rating`, `total_ratings`, `is_active`.
2. Does it need an API?	The service's contract: what other services and the apps can call.	Yes. `GET` and `POST /users/{user_id}`; `GET` and `POST /launderers/{launderer_id}/rating`.
3. What latency is acceptable?	The speed target, which drives database and caching choices.	Reads very fast: under 150 ms at p99. Writes can be slower: under 500 ms.

User Service: consistency, storage, traffic

Question	What it decides	User Service
4. How consistent should it be?	Must a write be readable everywhere immediately (strong), or is a short delay fine (eventual)?	
5. How much storage is needed?	A capacity estimate, to pick a database plan and prepare for scaling.	
6. How many reads vs. writes?	The traffic pattern, which drives the database and caching strategy.	

User Service: consistency, storage, traffic

Question	What it decides	User Service
4. How consistent should it be?	Must a write be readable everywhere immediately (strong), or is a short delay fine (eventual)?	Eventual. A profile update can take a few seconds to propagate; not every customer needs a new rating right away.
5. How much storage is needed?	A capacity estimate, to pick a database plan and prepare for scaling.	Very little at first: 10k users $\times$ 5 KB $=$ 50 MB, trivial for a modern database.
6. How many reads vs. writes?	The traffic pattern, which drives the database and caching strategy.	Highly read-heavy, about 100:1. Profiles are read on every app open; ratings are read constantly and written once per job.

Is eventual consistency always fine?

A customer changes `saved_address`, then requests a pickup straight away.

Is eventual consistency always fine?

A customer changes `saved_address`, then requests a pickup straight away.

- The Job Service reads the profile before the update has propagated.
- The launderer goes to the old address.

Is eventual consistency always fine?

A customer changes `saved_address`, then requests a pickup straight away.

- The Job Service reads the profile before the update has propagated.
- The launderer goes to the old address.

Fields the next request depends on need read-your-writes consistency: read them from the primary database, not a replica or the cache, right after an update.

User Service: database, scaling, cache

Question	What it decides	User Service
7. Relational or flexible data?	SQL for structured, relational data; NoSQL for flexible documents that scale more easily.	
8. How will the service scale?	From 10k to 10M users: a bigger server (vertical) or more identical servers (horizontal).	
9. Does it need a cache?	A fast in-memory store (e.g., Redis) in front of the database for frequently read data.	

User Service: database, scaling, cache

Question	What it decides	User Service
7. Relational or flexible data?	SQL for structured, relational data; NoSQL for flexible documents that scale more easily.	NoSQL (e.g., DynamoDB, MongoDB). A profile is self-contained, one document suits a read-heavy load, and the schema can grow.
8. How will the service scale?	From 10k to 10M users: a bigger server (vertical) or more identical servers (horizontal).	Horizontally: identical instances behind a load balancer. Possible because the service is stateless.
9. Does it need a cache?	A fast in-memory store (e.g., Redis) in front of the database for frequently read data.	Yes. Read-heavy and rarely changing, so check the cache first and skip the database: faster responses, protected database.

User Service: state, limits, metrics

Question	What it decides	User Service
10. Is it stateless?	Does it remember anything between requests, or get everything with each request?	
11. Does it need rate limiting?	A security and cost control: cap requests per user or IP address.	
12. What key metrics should it log?	You can't fix what you can't see.	

User Service: state, limits, metrics

Question	What it decides	User Service
10. Is it stateless?	Does it remember anything between requests, or get everything with each request?	Completely stateless. Each request carries an Identity Provider token, so any server can handle any request.
11. Does it need rate limiting?	A security and cost control: cap requests per user or IP address.	Yes, on writes, so a script can't change an address 1,000 times a second. For example, 10 profile updates per minute.
12. What key metrics should it log?	You can't fix what you can't see.	Request latency at p99, error rate, and cache hit rate.

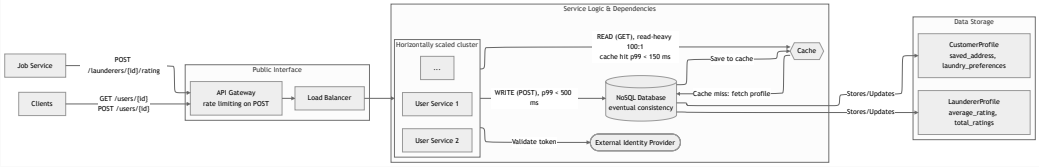
User Service: dependencies

Question	What it decides	User Service
13. External dependencies? Limits?	What it relies on, and what happens when each dependency fails.	

User Service: dependencies

Question	What it decides	User Service
13. External dependencies? Limits?	What it relies on, and what happens when each dependency fails.	Three. If the **database** is down, nothing can be read or written. If the **cache** is down, reads fall back to the database: slower, but working. If the **Identity Provider** is down, tokens can't be validated and every request fails authentication.

User Service component design



Homework 2: component-level design (individual, collaboration permitted)

Draft a component-level design for the Machine Service:

1. Review the finished high-level design.
2. Answer the thirteen component architecture questions.
3. Create a rough design diagram. Mermaid is a useful tool: `https://mermaid.js.org/`

Full handout: `assignments/hw2-component-design/HOMEWORK.md`